

SOS-BESOIN

Plateforme web transactionnelle de demandes urgentes

Cours	420-951-MA - Developpement Web 4 (Hiver 2026)
Laboratoire	Lab 2 - Server Actions, API Routes & Panier
Livrable	Livrable 1 - Document PDF
Equipe	SuperMoms
Sujet du projet	SOS-BESOIN - marketplace de demandes urgentes
Date de remise	27 avril 2026

Membres de l'equipe

Marina Kamel	Server Actions + Zod + Formulaires (Epic A)
Asma Ajroudi	API Routes + Postman (Epic B.1)
Sonia Corbin	Requetes Prisma avancees + Pagination + Agregation (Epic B.2/B.3)
Sonia Mhimdi	Panier client persistant + UX (Epic C)

Lien vers le depot GitHub (public, cliquable) :

<https://github.com/MarinaKamel-coder/sos-besoin>

Table des matieres

1. Presentation du projet
2. Lien vers le depot GitHub
3. Server Actions implementees
4. API Routes RESTful
5. Captures d'ecran - Tests Postman
6. Fonctionnement du panier
7. Captures d'ecran - Interface utilisateur
8. Bonus - Pagination par curseur

1. Presentation du projet

SOS-BESOIN est une plateforme web transactionnelle qui permet a des **clients** de publier des demandes urgentes et a des **prestataires** de soumettre des offres. Le client peut accepter une offre et confirmer la reservation via un flux de paiement (Stripe en mode test) avec integrite transactionnelle (ACID) gere par Prisma.

Stack technique

Framework	Next.js 16.2 (App Router)
Langage	TypeScript
Base de donnees	PostgreSQL (Neon)
ORM	Prisma 7.7
Authentification	Clerk (@clerk/nextjs ^7.0)
Validation	Zod
Style	Tailwind CSS v4
Paieement	Stripe (test mode) - prevu

Modele de donnees principal

Le schema Prisma definit les entites suivantes : User, Profile, Category, ServiceRequest, RequestCategory (table de jointure N-N), Offer, Booking, Payment et AdminAction. Les modeles ServiceRequest et Offer integrent un champ version (Int) qui implemente un verrouillage optimiste.

2. Lien vers le depot GitHub

Le depot est **public** et accessible a l'adresse suivante (URL cliquable) :

<https://github.com/MarinaKamel-coder/sos-besoin>

Contributions individuelles (git log). Chaque membre de l'equipe a des commits visibles dans l'historique du projet.

3. Server Actions implementees

Les Server Actions (Next.js App Router) sont declarees avec "use server" et appelees directement depuis les composants client via useActionState. Toutes les entrees sont validees par un schema Zod avant ecriture en base.

3.1 Profil utilisateur (src/action/userActions.ts)

Action	Role	Schema Zod
getMyProfileAction()	Recupere le profil de l'utilisateur Clerk connecte.	-
updateProfileAction(state, formData)	Upsert du profil utilisateur connecte.	profileUpdateSchema
deleteAccountAction()	Supprime le compte utilisateur (cascade Prisma).	-

Schema Zod : profileUpdateSchema

```
name : string min(2) max(50) optional
bio : string max(500) optional
city : string min(2) optional
phone : regex 10 a 15 chiffres optional
```

3.2 Demandes (src/action/requestActions.ts)

Action	Role	Schema Zod
getRequestsAction(filters?)	Liste les demandes (avec categories et nombre d'offres).	
createRequestAction(state, formData)	Crée une demande pour le client connecte.	requestCreateSchema
updateRequestAction(state, formData)	Mise a jour avec verrou optimiste (version).	requestUpdateSchema
deleteRequestAction(id)	Supprime une demande dont l'utilisateur est propriétaire.	

Schema Zod : requestCreateSchema

```
title : string min(5) max(100)
description : string min(10) max(1000)
neededAt : date (doit etre dans le futur)
location : string min(2) trimmed
categoryId : string min(1)
```

Schema Zod : requestUpdateSchema

```
= requestCreateSchema.partial() etendu avec :
id : string min(1) (requis)
version : int positif (requis pour le verrou optimiste)
```

Verrouillage optimiste. updateRequestAction utilise prisma.serviceRequest.updateMany avec un filtre where: id, version et un increment de version. Si result.count est 0, on retourne un message de conflit.

3.3 Offres (src/action/offerActions.ts)

Action	Role	Schema Zod
getOffersByRequestAction(requestId)	Ajoute les offres d'une demande, tri par prix.	-
createOfferAction(state, formData)	Cree une offre. Unicite (requestId, providerId).	offerCreateSchema
updateOfferAction(state, formData)	Met a jour prix/message avec verrou optimiste.	offerUpdateSchema
deleteOfferAction(id, version)	Retire une offre du prestataire (verrou optimiste).	(version)

Schema Zod : offerCreateSchema

```
price : number int positive (en cents)
message : string min(10) max(500)
requestId : string cuid
```

4. API Routes RESTful

Les API Routes (Next.js App Router) exposent les ressources principales en REST. Chaque endpoint valide les payloads avec Zod et renvoie des codes HTTP standards.

Endpoint	Methodes	Description
<code>/api/service-requests</code>	GET, POST	GET: liste les demandes. POST: cree une demande validee par Zod
<code>/api/service-requests/[id]</code>	GET, PUT, PATCH, DELETE	GET: detail. PUT: remplacement complet. PATCH: partiel. DELETE:
<code>/api/service-requests/[id]/offers</code>	GET, POST	GET: liste les offres d'une demande. POST: cree une offre.
<code>/api/offers/[id]</code>	GET, PATCH, PUT, DELETE	PUT: change le statut. DELETE: refusee si offre liee a un booking

4.1 Exemple : POST `/api/service-requests`

Requete (body JSON)

```
{
  "title": "Plomberie urgente cuisine",
  "description": "Fuite sous l evier",
  "neededAt": "2026-05-10T09:00:00.000Z",
  "location": "Longueuil",
  "clientId": "clx123abc",
  "categoryId": "clxcat999"
}
```

Reponse 201 Created

```
{
  "id": "clxreq456",
  "title": "Plomberie urgente cuisine",
  "status": "OPEN",
  "version": 1,
  "client": { "email": "...", "name": "..." },
  "categories": [{ "category": { "name": "Plomberie" } }]
}
```

Reponse 400 (validation Zod echouee)

```
{
  "error": "Validation echouee",
  "details": { "title": ["min 5 caracteres"] }
}
```

4.2 Exemple : PATCH `/api/offers/[id]`

```
{ "status": "ACCEPTED" }
```

Reponse 200 OK

```
{
  "id": "clxoffer123",
  "price": 12000,
  "status": "ACCEPTED",
}
```

```
"version": 2  
}
```


5. Captures d'ecran - Tests Postman

Une collection Postman complete est versionnee dans le depot a l'emplacement DOCS/postman/sos-besoin-api.postman_collection.json, accompagnee de l'environnement sos-besoin-local.postman_environment.json.

[Espace reserve a la capture d'ecran Postman]

[Espace reserve a la capture d'ecran Postman]

[Espace reserve a la capture d'ecran Postman]

[Espace reserve a la capture d'ecran Postman]

[Espace reserve a la capture d'ecran Postman]

6. Fonctionnement du panier

Section en attente. L'Epic C (Panier client persistant + UX) est realisee par **Sonia Mhimdi**. Cette section sera completee des que la livraison sera disponible.

- Description pas a pas du flux : ajout, modification, suppression, total.
- Diagramme de flux du processus de panier.
- Persistance cote client (localStorage) ou cote serveur (Cart / CartItem).
- Server Actions associees + schemas Zod.
- Composants UI : CartButton, CartDrawer, CartItem, CartSummary.

[Diagramme du flux panier - sera fourni avec la livraison de l'Epic C]

7. Captures d'ecran - Interface utilisateur

Les captures suivantes illustrent les ecrans principaux. La pagination est implementee dans `src/components/Pagination.tsx` et integree a la page `src/app/service-requests/page.tsx`. La liste utilise `getPaginatedServiceRequests()` definie dans `src/lib/requetes/serviceRequests.ts`.

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

[Espace reserve a la capture d'ecran]

8. Bonus - Pagination par curseur

Le bonus retenu est l'implementation d'une **pagination par curseur** (cursor-based pagination) en plus de la pagination par offset deja en place. Les deux approches coexistent dans le projet et utilisent les memes filtres.

8.1 Implementation

- Fonction `getCursorPaginatedServiceRequests()` ajoutée dans `src/lib/requetes/serviceRequests.ts`.
- Page de demo dediee : `src/app/service-requests/cursor/page.tsx`.
- URL : `/service-requests/cursor?cursor=lastId&q=...&status=OPEN`
- Page existante `/service-requests` conservee en mode **offset**.

Extrait du code Prisma (curseur)

```
const items = await prisma.serviceRequest.findMany({
  where,
  take: take + 1, // +1 pour detecter hasMore
  ...(params.cursor ? {
    cursor: { id: params.cursor },
    skip: 1, // ne PAS reinclure l'item du curseur
  } : {}),
  orderBy: [{ createdAt: "desc" }, { id: "desc" }],
  include: { client: true, categories: ..., _count: ... },
});
```

Astuce. On demande `take + 1` elements : si on en recoit `take + 1`, on sait qu'il existe une page suivante (`hasMore = true`) et le `nextCursor` est l'id du dernier element retourne. Le `orderBy` inclut un tie-breaker stable sur `id` pour eviter les sauts d'elements lors d'egalites sur `createdAt`.

8.2 Comparaison Offset vs Curseur

Critere	Offset (skip / take)	Curseur (cursor / take)
Requete SQL generee	OFFSET N LIMIT M	WHERE id apres cursor + LIMIT M
Performance gros volumes	Degradé avec N (scan + rejet des N premieres lignes)	Stable : utilise un index unique sur id.
Acces a une page N	Direct (page=42).	Impossible : navigation sequentielle uniquement.
Retour en arriere	Trivial (page--).	Possible mais demande une pile de curseurs cote client.
Insertion concurrente	Risque de duplication ou saut d'elements.	Robuste : un nouvel element n'affecte pas la suite.
Affichage du total	Naturel (count() en parallele).	Plus couteux : count() reste necessaire.
Implementation	Simple : skip / take.	Subtile : tie-breaker + take+1 + skip:1.
Cas d'usage	Tableaux d'admin, navigation arbitraire.	Feeds, scroll infini, gros volumes.

8.3 Conclusion

Pour SOS-BESOIN, la pagination par **offset** reste pertinente sur /service-requests car elle permet d'afficher un total et de sauter directement a une page donnee. La pagination par **curseur**, exposee sur /service-requests/cursor, devient le meilleur choix des que la liste depasse plusieurs milliers d'enregistrements ou pour un futur scroll infini : Postgres exploite directement l'index unique sur id au lieu de scanner et de jeter des lignes.